100 Medium to Hard LeetCode Data Structures Problems with Java Solutions

Table of Contents

1. Arrays

Problem 1: Product of Array Except Self (Medium)

Description: Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all elements except `nums[i]`.

Solution:

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];

        result[0] = 1;
        for (int i = 1; i &lt; n; i++) {
            result[i] = result[i - 1] * nums[i - 1];
        }

        int rightProduct = 1;
        for (int i = n - 1; i &gt;= 0; i--) {
            result[i] *= rightProduct;
            rightProduct *= nums[i];
        }

        return result;
    }
}
```

Problem 2: 3Sum (Medium)

Description: Given an integer array `nums`, return all triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `i != k`, and `j != k`, and `nums[i] + nums[j] + nums[k] == 0`.

Solution:

```
class Solution {
    public List&lt;List&lt;Integer&gt;&gt; threeSum(int[] nums) {
        List&lt;List&lt;Integer&gt;&gt; result = new ArrayList&lt;&gt;();
        Arrays.sort(nums);

        for (int i = 0; i &lt; nums.length - 2; i++) {
            if (i &gt; 0 &amp;&amp; nums[i] == nums[i - 1]) continue;
```

```
            int left = i + 1, right = nums.length - 1;
            while (left &lt; right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (sum == 0) {
                    result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                    while (left &lt; right &amp;&amp; nums[left] == nums[left + 1]) left++;
                    while (left &lt; right &amp;&amp; nums[right] == nums[right - 1]) right--;
                    left++;
                    right--;
                } else if (sum &lt; 0) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return result;
    }
}
```

Problem 3: Container With Most Water (Medium)

Description: Given `n` non-negative integers where each represents a point at coordinate `(i, ai)`, find two lines that together with the x-axis form a container with the most water.

Solution:

```
class Solution {
    public int maxArea(int[] height) {
        int left = 0, right = height.length - 1;
        int maxArea = 0;

        while (left &lt; right) {
            int h = Math.min(height[left], height[right]);
            maxArea = Math.max(maxArea, h * (right - left));

            if (height[left] &lt; height[right]) {
                left++;
            } else {
                right--;
            }
        }

        return maxArea;
    }
}
```

Problem 4: Search in Rotated Sorted Array (Medium)

Description: Given a rotated sorted array and a target value, return the index if the target is found, otherwise return -1.

Solution:

```
class Solution {
    public int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;

        while (left &lt;= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target) return mid;

            if (nums[left] &lt;= nums[mid]) {
                if (target &gt;= nums[left] &amp;&amp; target &lt; nums[mid]) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
```

```
                }
            } else {
                if (target &gt; nums[mid] &amp;&amp; target &lt;= nums[right]) {
                    left = mid + 1;
                } else {
                    right = mid - 1;
                }
            }
        }

        return -1;
    }
}
```

Problem 5: Find First and Last Position of Element (Medium)

Description: Given a sorted array of integers, find the starting and ending position of a given target value.

Solution:

```
class Solution {
    public int[] searchRange(int[] nums, int target) {
        int[] result = {-1, -1};
        result[0] = findFirst(nums, target);
        result[1] = findLast(nums, target);
        return result;
    }

    private int findFirst(int[] nums, int target) {
        int left = 0, right = nums.length - 1, result = -1;
        while (left &lt;= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                result = mid;
                right = mid - 1;
            } else if (nums[mid] &lt; target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return result;
    }

    private int findLast(int[] nums, int target) {
        int left = 0, right = nums.length - 1, result = -1;
        while (left &lt;= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                result = mid;
                left = mid + 1;
            } else if (nums[mid] &lt; target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return result;
    }
}
```

Problem 6: Spiral Matrix (Medium)

Description: Given an `m x n` matrix, return all elements in spiral order.

Solution:

```
class Solution {
    public List&lt;Integer&gt; spiralOrder(int[][] matrix) {
        List&lt;Integer&gt; result = new ArrayList&lt;&gt;();
        if (matrix.length == 0) return result;

        int top = 0, bottom = matrix.length - 1;
        int left = 0, right = matrix[0].length - 1;

        while (top &lt;= bottom &amp;&amp; left &lt;= right) {
            for (int i = left; i &lt;= right; i++) {
                result.add(matrix[top][i]);
            }
            top++;

            for (int i = top; i &lt;= bottom; i++) {
                result.add(matrix[i][right]);
            }
            right--;

            if (top &lt;= bottom) {
                for (int i = right; i &gt;= left; i--) {
                    result.add(matrix[bottom][i]);
                }
                bottom--;
            }

            if (left &lt;= right) {
                for (int i = bottom; i &gt;= top; i--) {
                    result.add(matrix[i][left]);
                }
                left++;
            }
        }

        return result;
    }
}
```

Problem 7: Rotate Image (Medium)

Description: You are given an `n x n` 2D matrix representing an image. Rotate the image by 90 degrees clockwise.

Solution:

```
class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;

        // Transpose
        for (int i = 0; i &lt; n; i++) {
            for (int j = i; j &lt; n; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }

        // Reverse each row
        for (int i = 0; i &lt; n; i++) {
            int left = 0, right = n - 1;
            while (left &lt; right) {
                int temp = matrix[i][left];
                matrix[i][left] = matrix[i][right];
```

```
                matrix[i][right] = temp;
                left++;
                right--;
            }
        }
    }
}
```

Problem 8: Longest Consecutive Sequence (Hard)

Description: Given an unsorted array of integers, return the length of the longest consecutive sequence.

Solution:

```
class Solution {
    public int longestConsecutive(int[] nums) {
        Set&lt;Integer&gt; numSet = new HashSet&lt;&gt;();
        for (int num : nums) {
            numSet.add(num);
        }

        int longest = 0;
        for (int num : numSet) {
            if (!numSet.contains(num - 1)) {
                int currentNum = num;
                int currentStreak = 1;

                while (numSet.contains(currentNum + 1)) {
                    currentNum++;
                    currentStreak++;
                }

                longest = Math.max(longest, currentStreak);
            }
        }

        return longest;
    }
}
```

Problem 9: Find Peak Element (Medium)

Description: A peak element is an element that is strictly greater than its neighbors. Find a peak element and return its index.

Solution:

```
class Solution {
    public int findPeakElement(int[] nums) {
        int left = 0, right = nums.length - 1;

        while (left &lt; right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] &gt; nums[mid + 1]) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }
}
```

Problem 10: Sort Colors (Medium)

Description: Given an array with `n` objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent.

Solution:

```
class Solution {
    public void sortColors(int[] nums) {
        int low = 0, mid = 0, high = nums.length - 1;

        while (mid &lt;= high) {
            if (nums[mid] == 0) {
                swap(nums, low, mid);
                low++;
                mid++;
            } else if (nums[mid] == 1) {
                mid++;
            } else {
                swap(nums, mid, high);
                high--;
            }
        }
    }

    private void swap(int[] nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
}
```

Problem 11: Subarray Sum Equals K (Medium)

Description: Given an array of integers and an integer `k`, find the total number of continuous subarrays whose sum equals to `k`.

Solution:

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        Map&lt;Integer, Integer&gt; prefixSum = new HashMap&lt;&gt;();
        prefixSum.put(0, 1);

        int sum = 0, count = 0;
        for (int num : nums) {
            sum += num;
            if (prefixSum.containsKey(sum - k)) {
                count += prefixSum.get(sum - k);
            }
            prefixSum.put(sum, prefixSum.getOrDefault(sum, 0) + 1);
        }

        return count;
    }
}
```

Problem 12: Maximum Product Subarray (Medium)

Description: Given an integer array, find a contiguous non-empty subarray with the largest product.

Solution:

```
class Solution {
    public int maxProduct(int[] nums) {
        int maxProd = nums[0];
        int currentMax = nums[0];
        int currentMin = nums[0];
```

```
        for (int i = 1; i &lt; nums.length; i++) {
            if (nums[i] &lt; 0) {
                int temp = currentMax;
                currentMax = currentMin;
                currentMin = temp;
            }

            currentMax = Math.max(nums[i], currentMax * nums[i]);
            currentMin = Math.min(nums[i], currentMin * nums[i]);

            maxProd = Math.max(maxProd, currentMax);
        }

        return maxProd;
    }
}
```

Problem 13: Next Permutation (Medium)

Description: Implement next permutation, which rearranges numbers into the lexicographically next greater permutation.

Solution:

```
class Solution {
    public void nextPermutation(int[] nums) {
        int i = nums.length - 2;
        while (i &gt;= 0 &amp;&amp; nums[i] &gt;= nums[i + 1]) {
            i--;
        }

        if (i &gt;= 0) {
            int j = nums.length - 1;
            while (nums[j] &lt;= nums[i]) {
                j--;
            }
            swap(nums, i, j);
        }

        reverse(nums, i + 1, nums.length - 1);
    }

    private void swap(int[] nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }

    private void reverse(int[] nums, int start, int end) {
        while (start &lt; end) {
            swap(nums, start, end);
            start++;
            end--;
        }
    }
}
```

Problem 14: Merge Intervals (Medium)

Description: Given an array of intervals, merge all overlapping intervals.

Solution:

```
class Solution {
    public int[][] merge(int[][] intervals) {
        if (intervals.length &lt;= 1) return intervals;

        Arrays.sort(intervals, (a, b) -&gt; Integer.compare(a[0], b[0]));
```

```
        List&lt;int[]&gt; result = new ArrayList&lt;&gt;();
        int[] currentInterval = intervals[0];
        result.add(currentInterval);

        for (int[] interval : intervals) {
            int currentEnd = currentInterval[1];
            int nextStart = interval[0];
            int nextEnd = interval[1];

            if (currentEnd &gt;= nextStart) {
                currentInterval[1] = Math.max(currentEnd, nextEnd);
            } else {
                currentInterval = interval;
                result.add(currentInterval);
            }
        }

        return result.toArray(new int[result.size()][]);
    }
}
```

Problem 15: First Missing Positive (Hard)

Description: Given an unsorted integer array, find the smallest missing positive integer.

Solution:

```
class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;

        for (int i = 0; i &lt; n; i++) {
            while (nums[i] &gt; 0 &amp;&amp; nums[i] &lt;= n &amp;&amp;
                   nums[nums[i] - 1] != nums[i]) {
                int temp = nums[nums[i] - 1];
                nums[nums[i] - 1] = nums[i];
                nums[i] = temp;
            }
        }

        for (int i = 0; i &lt; n; i++) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }

        return n + 1;
    }
}
```

2. Linked Lists

Problem 16: Add Two Numbers (Medium)

Description: You are given two non-empty linked lists representing two non-negative integers. Add the two numbers and return the sum as a linked list.

Solution:

```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode current = dummy;
        int carry = 0;
```

```
        while (l1 != null || l2 != null || carry != 0) {
            int val1 = (l1 != null) ? l1.val : 0;
            int val2 = (l2 != null) ? l2.val : 0;

            int sum = val1 + val2 + carry;
            carry = sum / 10;
            current.next = new ListNode(sum % 10);
            current = current.next;

            if (l1 != null) l1 = l1.next;
            if (l2 != null) l2 = l2.next;
        }

        return dummy.next;
    }
}
```

Problem 17: Reverse Linked List II (Medium)

Description: Reverse a linked list from position `m` to `n`.

Solution:

```
class Solution {
    public ListNode reverseBetween(ListNode head, int left, int right) {
        if (head == null || left == right) return head;

        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode prev = dummy;

        for (int i = 0; i &lt; left - 1; i++) {
            prev = prev.next;
        }

        ListNode current = prev.next;
        ListNode next = null;

        for (int i = 0; i &lt; right - left; i++) {
            next = current.next;
            current.next = next.next;
            next.next = prev.next;
            prev.next = next;
        }

        return dummy.next;
    }
}
```

Problem 18: Linked List Cycle II (Medium)

Description: Given a linked list, return the node where the cycle begins. If there is no cycle, return null.

Solution:

```
class Solution {
    public ListNode detectCycle(ListNode head) {
        if (head == null) return null;

        ListNode slow = head, fast = head;
        boolean hasCycle = false;

        while (fast != null &amp;&amp; fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                hasCycle = true;
                break;
```

```
            }
        }

        if (!hasCycle) return null;

        slow = head;
        while (slow != fast) {
            slow = slow.next;
            fast = fast.next;
        }

        return slow;
    }
}
```

Problem 19: Remove Nth Node From End (Medium)

Description: Remove the nth node from the end of a linked list.

Solution:

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode fast = dummy, slow = dummy;

        for (int i = 0; i &lt;= n; i++) {
            fast = fast.next;
        }

        while (fast != null) {
            fast = fast.next;
            slow = slow.next;
        }

        slow.next = slow.next.next;
        return dummy.next;
    }
}
```

Problem 20: Reorder List (Medium)

Description: Given a singly linked list, reorder it to: L0 → Ln → L1 → Ln-1 → L2 → Ln-2 → ...

Solution:

```
class Solution {
    public void reorderList(ListNode head) {
        if (head == null || head.next == null) return;

        // Find middle
        ListNode slow = head, fast = head;
        while (fast.next != null &amp;&amp; fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        // Reverse second half
        ListNode secondHalf = reverse(slow.next);
        slow.next = null;

        // Merge two halves
        ListNode first = head, second = secondHalf;
        while (second != null) {
            ListNode temp1 = first.next;
            ListNode temp2 = second.next;
            first.next = second;
```

```
            second.next = temp1;
            first = temp1;
            second = temp2;
        }
    }

    private ListNode reverse(ListNode head) {
        ListNode prev = null;
        while (head != null) {
            ListNode next = head.next;
            head.next = prev;
            prev = head;
            head = next;
        }
        return prev;
    }
}
```

Problem 21: Merge K Sorted Lists (Hard)

Description: Merge k sorted linked lists and return it as one sorted list.

Solution:

```
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        if (lists == null || lists.length == 0) return null;

        PriorityQueue&lt;ListNode&gt; pq = new PriorityQueue&lt;&gt;((a, b) -&gt; a.val - b.val);

        for (ListNode node : lists) {
            if (node != null) {
                pq.offer(node);
            }
        }

        ListNode dummy = new ListNode(0);
        ListNode current = dummy;

        while (!pq.isEmpty()) {
            ListNode node = pq.poll();
            current.next = node;
            current = current.next;
            if (node.next != null) {
                pq.offer(node.next);
            }
        }

        return dummy.next;
    }
}
```

Problem 22: Copy List with Random Pointer (Medium)

Description: A linked list is given such that each node contains an additional random pointer. Clone the list.

Solution:

```
class Solution {
    public Node copyRandomList(Node head) {
        if (head == null) return null;

        Map&lt;Node, Node&gt; map = new HashMap&lt;&gt;();
        Node current = head;

        // First pass: create all nodes
        while (current != null) {
            map.put(current, new Node(current.val));
```

```
            current = current.next;
        }

        // Second pass: assign next and random pointers
        current = head;
        while (current != null) {
            map.get(current).next = map.get(current.next);
            map.get(current).random = map.get(current.random);
            current = current.next;
        }

        return map.get(head);
    }
}
```

Problem 23: Palindrome Linked List (Medium)

Description: Determine if a singly linked list is a palindrome.

Solution:

```
class Solution {
    public boolean isPalindrome(ListNode head) {
        if (head == null || head.next == null) return true;

        // Find middle
        ListNode slow = head, fast = head;
        while (fast.next != null &amp;amp;&amp;amp; fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        // Reverse second half
        ListNode secondHalf = reverse(slow.next);

        // Compare
        ListNode p1 = head, p2 = secondHalf;
        while (p2 != null) {
            if (p1.val != p2.val) return false;
            p1 = p1.next;
            p2 = p2.next;
        }

        return true;
    }

    private ListNode reverse(ListNode head) {
        ListNode prev = null;
        while (head != null) {
            ListNode next = head.next;
            head.next = prev;
            prev = head;
            head = next;
        }
        return prev;
    }
}
```

Problem 24: Sort List (Medium)

Description: Sort a linked list in O(n log n) time and constant space complexity.

Solution:

```
class Solution {
    public ListNode sortList(ListNode head) {
        if (head == null || head.next == null) return head;
```

```
        ListNode mid = getMid(head);
        ListNode left = sortList(head);
        ListNode right = sortList(mid);

        return merge(left, right);
    }

    private ListNode getMid(ListNode head) {
        ListNode slow = head, fast = head, prev = null;
        while (fast != null &amp;amp;&amp;amp; fast.next != null) {
            prev = slow;
            slow = slow.next;
            fast = fast.next.next;
        }
        if (prev != null) prev.next = null;
        return slow;
    }

    private ListNode merge(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode current = dummy;

        while (l1 != null &amp;amp;&amp;amp; l2 != null) {
            if (l1.val &amp;lt; l2.val) {
                current.next = l1;
                l1 = l1.next;
            } else {
                current.next = l2;
                l2 = l2.next;
            }
            current = current.next;
        }

        current.next = (l1 != null) ? l1 : l2;
        return dummy.next;
    }
}
```

Problem 25: Intersection of Two Linked Lists (Medium)

Description: Find the node at which two singly linked lists intersect.

Solution:

```
class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        if (headA == null || headB == null) return null;

        ListNode pA = headA, pB = headB;

        while (pA != pB) {
            pA = (pA == null) ? headB : pA.next;
            pB = (pB == null) ? headA : pB.next;
        }

        return pA;
    }
}
```

Problem 26: Flatten a Multilevel Doubly Linked List (Medium)

Description: Flatten a multilevel doubly linked list so that all nodes appear in a single-level list.

Solution:

```
class Solution {
    public Node flatten(Node head) {
        if (head == null) return null;
```

```
        Node current = head;
        while (current != null) {
            if (current.child != null) {
                Node next = current.next;
                Node child = flatten(current.child);

                current.next = child;
                child.prev = current;
                current.child = null;

                while (current.next != null) {
                    current = current.next;
                }

                if (next != null) {
                    current.next = next;
                    next.prev = current;
                }
            }
            current = current.next;
        }

        return head;
    }
}
```

Problem 27: Swap Nodes in Pairs (Medium)

Description: Given a linked list, swap every two adjacent nodes and return its head.

Solution:

```
class Solution {
    public ListNode swapPairs(ListNode head) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode current = dummy;

        while (current.next != null &amp;&amp; current.next.next != null) {
            ListNode first = current.next;
            ListNode second = current.next.next;

            first.next = second.next;
            current.next = second;
            second.next = first;

            current = first;
        }

        return dummy.next;
    }
}
```

3. Stacks & Queues

Problem 28: Valid Parentheses (Medium)

Description: Given a string containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

Solution:

```
class Solution {
    public boolean isValid(String s) {
        Stack&lt;Character&gt; stack = new Stack&lt;&gt;();
        Map&lt;Character, Character&gt; map = new HashMap&lt;&gt;();
```

```
        map.put(')', '(');
        map.put('}', '{');
        map.put(']', '[');

        for (char c : s.toCharArray()) {
            if (map.containsKey(c)) {
                if (stack.isEmpty() || stack.pop() != map.get(c)) {
                    return false;
                }
            } else {
                stack.push(c);
            }
        }

        return stack.isEmpty();
    }
}
```

Problem 29: Daily Temperatures (Medium)

Description: Given an array of daily temperatures, return an array such that answer[i] is the number of days you have to wait until a warmer temperature.

Solution:

```
class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int n = temperatures.length;
        int[] result = new int[n];
        Stack&lt;Integer&gt; stack = new Stack&lt;&gt;();

        for (int i = 0; i &lt; n; i++) {
            while (!stack.isEmpty() &amp;&amp; temperatures[i] &gt; temperatures[stack.peek()]) {
                int idx = stack.pop();
                result[idx] = i - idx;
            }
            stack.push(i);
        }

        return result;
    }
}
```

Problem 30: Largest Rectangle in Histogram (Hard)

Description: Given an array of integers representing the histogram's bar height, find the area of largest rectangle.

Solution:

```
class Solution {
    public int largestRectangleArea(int[] heights) {
        Stack&lt;Integer&gt; stack = new Stack&lt;&gt;();
        int maxArea = 0;
        int n = heights.length;

        for (int i = 0; i &lt;= n; i++) {
            int h = (i == n) ? 0 : heights[i];

            while (!stack.isEmpty() &amp;&amp; h &lt; heights[stack.peek()]) {
                int height = heights[stack.pop()];
                int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                maxArea = Math.max(maxArea, height * width);
            }
            stack.push(i);
        }

        return maxArea;
```

```
    }
}
```

Problem 31: Min Stack (Medium)

Description: Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Solution:

```
class MinStack {
    private Stack&lt;Integer&gt; stack;
    private Stack&lt;Integer&gt; minStack;

    public MinStack() {
        stack = new Stack&lt;&gt;();
        minStack = new Stack&lt;&gt;();
    }

    public void push(int val) {
        stack.push(val);
        if (minStack.isEmpty() || val &lt;= minStack.peek()) {
            minStack.push(val);
        }
    }

    public void pop() {
        int val = stack.pop();
        if (val == minStack.peek()) {
            minStack.pop();
        }
    }

    public int top() {
        return stack.peek();
    }

    public int getMin() {
        return minStack.peek();
    }
}
```

Problem 32: Decode String (Medium)

Description: Given an encoded string, return its decoded string. The encoding rule is: k[encoded_string].

Solution:

```
class Solution {
    public String decodeString(String s) {
        Stack&lt;Integer&gt; countStack = new Stack&lt;&gt;();
        Stack&lt;String&gt; stringStack = new Stack&lt;&gt;();
        String current = "";
        int k = 0;

        for (char c : s.toCharArray()) {
            if (Character.isDigit(c)) {
                k = k * 10 + (c - '0');
            } else if (c == '[') {
                countStack.push(k);
                stringStack.push(current);
                current = "";
                k = 0;
            } else if (c == ']') {
                StringBuilder decoded = new StringBuilder(stringStack.pop());
                int count = countStack.pop();
                for (int i = 0; i &lt; count; i++) {
                    decoded.append(current);
                }
```

```
                current = decoded.toString();
            } else {
                current += c;
            }
        }

        return current;
    }
}
```

Problem 33: Simplify Path (Medium)

Description: Given an absolute path for a file, simplify it to the canonical path.

Solution:

```
class Solution {
    public String simplifyPath(String path) {
        Stack&lt;String&gt; stack = new Stack&lt;&gt;();
        String[] components = path.split("/");

        for (String component : components) {
            if (component.equals("..")) {
                if (!stack.isEmpty()) {
                    stack.pop();
                }
            } else if (!component.equals("") &amp;&amp; !component.equals(".")) {
                stack.push(component);
            }
        }

        StringBuilder result = new StringBuilder();
        for (String dir : stack) {
            result.append("/").append(dir);
        }

        return result.length() &gt; 0 ? result.toString() : "/";
    }
}
```

Problem 34: Evaluate Reverse Polish Notation (Medium)

Description: Evaluate the value of an arithmetic expression in Reverse Polish Notation.

Solution:

```
class Solution {
    public int evalRPN(String[] tokens) {
        Stack&lt;Integer&gt; stack = new Stack&lt;&gt;();

        for (String token : tokens) {
            if (token.equals("+")) {
                stack.push(stack.pop() + stack.pop());
            } else if (token.equals("-")) {
                int b = stack.pop();
                int a = stack.pop();
                stack.push(a - b);
            } else if (token.equals("*")) {
                stack.push(stack.pop() * stack.pop());
            } else if (token.equals("/")) {
                int b = stack.pop();
                int a = stack.pop();
                stack.push(a / b);
            } else {
                stack.push(Integer.parseInt(token));
            }
        }
```

```
        return stack.pop();
    }
}
```

Problem 35: Next Greater Element II (Medium)

Description: Given a circular array, find the next greater number for every element.

Solution:

```
class Solution {
    public int[] nextGreaterElements(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        Arrays.fill(result, -1);
        Stack&lt;Integer&gt; stack = new Stack&lt;&gt;();

        for (int i = 0; i &lt; 2 * n; i++) {
            int num = nums[i % n];
            while (!stack.isEmpty() &amp;&amp; nums[stack.peek()] &lt; num) {
                result[stack.pop()] = num;
            }
            if (i &lt; n) {
                stack.push(i);
            }
        }

        return result;
    }
}
```

Problem 36: Sliding Window Maximum (Hard)

Description: Given an array and a sliding window of size k, find the maximum value in each window.

Solution:

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        if (nums.length == 0 || k == 0) return new int[0];

        int n = nums.length;
        int[] result = new int[n - k + 1];
        Deque&lt;Integer&gt; deque = new ArrayDeque&lt;&gt;();

        for (int i = 0; i &lt; n; i++) {
            // Remove elements outside window
            while (!deque.isEmpty() &amp;&amp; deque.peek() &lt; i - k + 1) {
                deque.poll();
            }

            // Remove smaller elements
            while (!deque.isEmpty() &amp;&amp; nums[deque.peekLast()] &lt; nums[i]) {
                deque.pollLast();
            }

            deque.offer(i);

            if (i &gt;= k - 1) {
                result[i - k + 1] = nums[deque.peek()];
            }
        }

        return result;
    }
}
```

Problem 37: Implement Queue using Stacks (Medium)

Description: Implement a queue using two stacks.

Solution:

```
class MyQueue {
    private Stack&lt;Integer&gt; inStack;
    private Stack&lt;Integer&gt; outStack;

    public MyQueue() {
        inStack = new Stack&lt;&gt;();
        outStack = new Stack&lt;&gt;();
    }

    public void push(int x) {
        inStack.push(x);
    }

    public int pop() {
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty()) {
                outStack.push(inStack.pop());
            }
        }
        return outStack.pop();
    }

    public int peek() {
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty()) {
                outStack.push(inStack.pop());
            }
        }
        return outStack.peek();
    }

    public boolean empty() {
        return inStack.isEmpty() &amp;&amp; outStack.isEmpty();
    }
}
```

4. Trees & Binary Search Trees

Problem 38: Binary Tree Level Order Traversal (Medium)

Description: Given a binary tree, return the level order traversal of its nodes' values.

Solution:

```
class Solution {
    public List&lt;List&lt;Integer&gt;&gt; levelOrder(TreeNode root) {
        List&lt;List&lt;Integer&gt;&gt; result = new ArrayList&lt;&gt;();
        if (root == null) return result;

        Queue&lt;TreeNode&gt; queue = new LinkedList&lt;&gt;();
        queue.offer(root);

        while (!queue.isEmpty()) {
            int levelSize = queue.size();
            List&lt;Integer&gt; currentLevel = new ArrayList&lt;&gt;();

            for (int i = 0; i &lt; levelSize; i++) {
                TreeNode node = queue.poll();
                currentLevel.add(node.val);

                if (node.left != null) queue.offer(node.left);
```

```
                if (node.right != null) queue.offer(node.right);
            }

            result.add(currentLevel);
        }

        return result;
    }
}
```

Problem 39: Validate Binary Search Tree (Medium)

Description: Given the root of a binary tree, determine if it is a valid binary search tree.

Solution:

```
class Solution {
    public boolean isValidBST(TreeNode root) {
        return validate(root, null, null);
    }

    private boolean validate(TreeNode node, Integer min, Integer max) {
        if (node == null) return true;

        if ((min != null &amp;&amp; node.val &lt;= min) || (max != null &amp;&amp; node.val &gt;= max)) {
            return false;
        }

        return validate(node.left, min, node.val) &amp;&amp;
               validate(node.right, node.val, max);
    }
}
```

Problem 40: Binary Tree Maximum Path Sum (Hard)

Description: Given a binary tree, find the maximum path sum. The path may start and end at any node.

Solution:

```
class Solution {
    private int maxSum = Integer.MIN_VALUE;

    public int maxPathSum(TreeNode root) {
        maxGain(root);
        return maxSum;
    }

    private int maxGain(TreeNode node) {
        if (node == null) return 0;

        int leftGain = Math.max(maxGain(node.left), 0);
        int rightGain = Math.max(maxGain(node.right), 0);

        int pathSum = node.val + leftGain + rightGain;
        maxSum = Math.max(maxSum, pathSum);

        return node.val + Math.max(leftGain, rightGain);
    }
}
```

Problem 41: Construct Binary Tree from Preorder and Inorder (Medium)

Description: Construct a binary tree from preorder and inorder traversal arrays.

Solution:

```
class Solution {
    private int preorderIndex = 0;
    private Map&lt;Integer, Integer&gt; inorderMap = new HashMap&lt;&gt;();

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        for (int i = 0; i &lt; inorder.length; i++) {
            inorderMap.put(inorder[i], i);
        }
        return build(preorder, 0, preorder.length - 1);
    }

    private TreeNode build(int[] preorder, int left, int right) {
        if (left &gt; right) return null;

        int rootVal = preorder[preorderIndex++];
        TreeNode root = new TreeNode(rootVal);

        int inorderIndex = inorderMap.get(rootVal);
        root.left = build(preorder, left, inorderIndex - 1);
        root.right = build(preorder, inorderIndex + 1, right);

        return root;
    }
}
```

Problem 42: Lowest Common Ancestor of Binary Tree (Medium)

Description: Find the lowest common ancestor of two nodes in a binary tree.

Solution:

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q) {
            return root;
        }

        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);

        if (left != null &amp;&amp; right != null) {
            return root;
        }

        return (left != null) ? left : right;
    }
}
```

Problem 43: Kth Smallest Element in BST (Medium)

Description: Given a binary search tree, find the kth smallest element.

Solution:

```
class Solution {
    private int count = 0;
    private int result = 0;

    public int kthSmallest(TreeNode root, int k) {
        inorder(root, k);
        return result;
```

```
    }

    private void inorder(TreeNode node, int k) {
        if (node == null) return;

        inorder(node.left, k);

        count++;
        if (count == k) {
            result = node.val;
            return;
        }

        inorder(node.right, k);
    }
}
```

Problem 44: Serialize and Deserialize Binary Tree (Hard)

Description: Design an algorithm to serialize and deserialize a binary tree.

Solution:

```
public class Codec {
    public String serialize(TreeNode root) {
        if (root == null) return "null";

        StringBuilder sb = new StringBuilder();
        Queue&lt;TreeNode&gt; queue = new LinkedList&lt;&gt;();
        queue.offer(root);

        while (!queue.isEmpty()) {
            TreeNode node = queue.poll();
            if (node == null) {
                sb.append("null,");
            } else {
                sb.append(node.val).append(",");
                queue.offer(node.left);
                queue.offer(node.right);
            }
        }

        return sb.toString();
    }

    public TreeNode deserialize(String data) {
        if (data.equals("null")) return null;

        String[] values = data.split(",");
        TreeNode root = new TreeNode(Integer.parseInt(values[0]));
        Queue&lt;TreeNode&gt; queue = new LinkedList&lt;&gt;();
        queue.offer(root);

        int i = 1;
        while (!queue.isEmpty()) {
            TreeNode node = queue.poll();

            if (!values[i].equals("null")) {
                node.left = new TreeNode(Integer.parseInt(values[i]));
                queue.offer(node.left);
            }
            i++;

            if (!values[i].equals("null")) {
                node.right = new TreeNode(Integer.parseInt(values[i]));
                queue.offer(node.right);
            }
            i++;
        }
```

```
        return root;
    }
}
```

Problem 45: Binary Tree Right Side View (Medium)

Description: Given a binary tree, return the values of nodes you can see from the right side.

Solution:

```
class Solution {
    public List&lt;Integer&gt; rightSideView(TreeNode root) {
        List&lt;Integer&gt; result = new ArrayList&lt;&gt;();
        if (root == null) return result;

        Queue&lt;TreeNode&gt; queue = new LinkedList&lt;&gt;();
        queue.offer(root);

        while (!queue.isEmpty()) {
            int levelSize = queue.size();

            for (int i = 0; i &lt; levelSize; i++) {
                TreeNode node = queue.poll();

                if (i == levelSize - 1) {
                    result.add(node.val);
                }

                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
        }

        return result;
    }
}
```

Problem 46: Flatten Binary Tree to Linked List (Medium)

Description: Flatten a binary tree to a linked list in-place using preorder traversal.

Solution:

```
class Solution {
    public void flatten(TreeNode root) {
        if (root == null) return;

        flatten(root.left);
        flatten(root.right);

        TreeNode right = root.right;
        root.right = root.left;
        root.left = null;

        TreeNode current = root;
        while (current.right != null) {
            current = current.right;
        }
        current.right = right;
    }
}
```

Problem 47: Populating Next Right Pointers (Medium)

Description: Populate each next pointer to point to its next right node in a perfect binary tree.

Solution:

```
class Solution {
    public Node connect(Node root) {
        if (root == null) return null;

        Queue&lt;Node&gt; queue = new LinkedList&lt;&gt;();
        queue.offer(root);

        while (!queue.isEmpty()) {
            int levelSize = queue.size();
            Node prev = null;

            for (int i = 0; i &lt; levelSize; i++) {
                Node node = queue.poll();

                if (prev != null) {
                    prev.next = node;
                }
                prev = node;

                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
        }

        return root;
    }
}
```

Problem 48: Binary Tree Zigzag Level Order (Medium)

Description: Given a binary tree, return the zigzag level order traversal.

Solution:

```
class Solution {
    public List&lt;List&lt;Integer&gt;&gt; zigzagLevelOrder(TreeNode root) {
        List&lt;List&lt;Integer&gt;&gt; result = new ArrayList&lt;&gt;();
        if (root == null) return result;

        Queue&lt;TreeNode&gt; queue = new LinkedList&lt;&gt;();
        queue.offer(root);
        boolean leftToRight = true;

        while (!queue.isEmpty()) {
            int levelSize = queue.size();
            List&lt;Integer&gt; currentLevel = new ArrayList&lt;&gt;();

            for (int i = 0; i &lt; levelSize; i++) {
                TreeNode node = queue.poll();

                if (leftToRight) {
                    currentLevel.add(node.val);
                } else {
                    currentLevel.add(0, node.val);
                }

                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }

            result.add(currentLevel);
            leftToRight = !leftToRight;
        }
```

```
        return result;
    }
}
```

Problem 49: Count Complete Tree Nodes (Medium)

Description: Given a complete binary tree, count the number of nodes.

Solution:

```
class Solution {
    public int countNodes(TreeNode root) {
        if (root == null) return 0;

        int leftHeight = getHeight(root.left);
        int rightHeight = getHeight(root.right);

        if (leftHeight == rightHeight) {
            return (1 &lt;&lt; leftHeight) + countNodes(root.right);
        } else {
            return (1 &lt;&lt; rightHeight) + countNodes(root.left);
        }
    }

    private int getHeight(TreeNode node) {
        int height = 0;
        while (node != null) {
            height++;
            node = node.left;
        }
        return height;
    }
}
```

Problem 50: Path Sum II (Medium)

Description: Given a binary tree and a sum, find all root-to-leaf paths where the path sum equals the given sum.

Solution:

```
class Solution {
    public List&lt;List&lt;Integer&gt;&gt; pathSum(TreeNode root, int targetSum) {
        List&lt;List&lt;Integer&gt;&gt; result = new ArrayList&lt;&gt;();
        List&lt;Integer&gt; path = new ArrayList&lt;&gt;();
        dfs(root, targetSum, path, result);
        return result;
    }

    private void dfs(TreeNode node, int remaining, List&lt;Integer&gt; path,
                     List&lt;List&lt;Integer&gt;&gt; result) {
        if (node == null) return;

        path.add(node.val);

        if (node.left == null &amp;&amp; node.right == null &amp;&amp; remaining == node.val) {
            result.add(new ArrayList&lt;&gt;(path));
        } else {
            dfs(node.left, remaining - node.val, path, result);
            dfs(node.right, remaining - node.val, path, result);
        }

        path.remove(path.size() - 1);
    }
}
```

Problem 51: Delete Node in BST (Medium)

Description: Given a BST and a key, delete the node with the given key.

Solution:

```
class Solution {
    public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null) return null;

        if (key &lt; root.val) {
            root.left = deleteNode(root.left, key);
        } else if (key &gt; root.val) {
            root.right = deleteNode(root.right, key);
        } else {
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;

            TreeNode minNode = findMin(root.right);
            root.val = minNode.val;
            root.right = deleteNode(root.right, root.val);
        }

        return root;
    }

    private TreeNode findMin(TreeNode node) {
        while (node.left != null) {
            node = node.left;
        }
        return node;
    }
}
```

Problem 52: Binary Tree Vertical Order Traversal (Medium)

Description: Return the vertical order traversal of a binary tree.

Solution:

```
class Solution {
    public List&lt;List&lt;Integer&gt;&gt; verticalOrder(TreeNode root) {
        List&lt;List&lt;Integer&gt;&gt; result = new ArrayList&lt;&gt;();
        if (root == null) return result;

        Map&lt;Integer, List&lt;Integer&gt;&gt; map = new TreeMap&lt;&gt;();
        Queue&lt;Pair&lt;TreeNode, Integer&gt;&gt; queue = new LinkedList&lt;&gt;();
        queue.offer(new Pair&lt;&gt;(root, 0));

        while (!queue.isEmpty()) {
            Pair&lt;TreeNode, Integer&gt; pair = queue.poll();
            TreeNode node = pair.getKey();
            int col = pair.getValue();

            map.putIfAbsent(col, new ArrayList&lt;&gt;());
            map.get(col).add(node.val);

            if (node.left != null) {
                queue.offer(new Pair&lt;&gt;(node.left, col - 1));
            }
            if (node.right != null) {
                queue.offer(new Pair&lt;&gt;(node.right, col + 1));
            }
        }

        result.addAll(map.values());
        return result;
```

```
    }
}
```

Problem 53: Convert Sorted Array to BST (Medium)

Description: Convert a sorted array to a height-balanced BST.

Solution:

```
class Solution {
    public TreeNode sortedArrayToBST(int[] nums) {
        return buildBST(nums, 0, nums.length - 1);
    }

    private TreeNode buildBST(int[] nums, int left, int right) {
        if (left &gt; right) return null;

        int mid = left + (right - left) / 2;
        TreeNode root = new TreeNode(nums[mid]);

        root.left = buildBST(nums, left, mid - 1);
        root.right = buildBST(nums, mid + 1, right);

        return root;
    }
}
```

Problem 54: Recover Binary Search Tree (Hard)

Description: Two elements of a BST are swapped by mistake. Recover the tree without changing its structure.

Solution:

```
class Solution {
    private TreeNode first = null;
    private TreeNode second = null;
    private TreeNode prev = null;

    public void recoverTree(TreeNode root) {
        inorder(root);

        int temp = first.val;
        first.val = second.val;
        second.val = temp;
    }

    private void inorder(TreeNode node) {
        if (node == null) return;

        inorder(node.left);

        if (prev != null &amp;&amp; prev.val &gt; node.val) {
            if (first == null) {
                first = prev;
            }
            second = node;
        }
        prev = node;

        inorder(node.right);
    }
}
```

Problem 55: Binary Tree Cameras (Hard)

Description: Given a binary tree, install cameras on nodes. Each camera can monitor its parent, itself, and immediate children. Return minimum cameras needed.

Solution:

```
class Solution {
    private int cameras = 0;

    public int minCameraCover(TreeNode root) {
        return (dfs(root) == 0) ? cameras + 1 : cameras;
    }

    private int dfs(TreeNode node) {
        if (node == null) return 1;

        int left = dfs(node.left);
        int right = dfs(node.right);

        if (left == 0 || right == 0) {
            cameras++;
            return 2;
        }

        if (left == 2 || right == 2) {
            return 1;
        }

        return 0;
    }
}
```

Problem 56: Sum of Distances in Tree (Hard)

Description: Given a tree, compute the sum of distances between each node and all other nodes.

Solution:

```
class Solution {
    private int[] count;
    private int[] result;
    private List&lt;Integer&gt;[] graph;

    public int[] sumOfDistancesInTree(int n, int[][] edges) {
        count = new int[n];
        result = new int[n];
        graph = new ArrayList[n];

        for (int i = 0; i &lt; n; i++) {
            graph[i] = new ArrayList&lt;&gt;();
        }

        for (int[] edge : edges) {
            graph[edge[0]].add(edge[1]);
            graph[edge[1]].add(edge[0]);
        }

        postorder(0, -1);
        preorder(0, -1);

        return result;
    }

    private void postorder(int node, int parent) {
        count[node] = 1;
        for (int child : graph[node]) {
            if (child != parent) {
                postorder(child, node);
```

```
                count[node] += count[child];
                result[node] += result[child] + count[child];
            }
        }
    }

    private void preorder(int node, int parent) {
        for (int child : graph[node]) {
            if (child != parent) {
                result[child] = result[node] - count[child] +
                                (count.length - count[child]);
                preorder(child, node);
            }
        }
    }
}
```

Problem 57: All Nodes Distance K in Binary Tree (Medium)

Description: Return all nodes that are distance K from the target node in a binary tree.

Solution:

```
class Solution {
    private Map&lt;TreeNode, TreeNode&gt; parent = new HashMap&lt;&gt;();

    public List&lt;Integer&gt; distanceK(TreeNode root, TreeNode target, int k) {
        buildParentMap(root, null);

        Queue&lt;TreeNode&gt; queue = new LinkedList&lt;&gt;();
        Set&lt;TreeNode&gt; visited = new HashSet&lt;&gt;();
        queue.offer(target);
        visited.add(target);

        int distance = 0;
        while (!queue.isEmpty()) {
            if (distance == k) {
                List&lt;Integer&gt; result = new ArrayList&lt;&gt;();
                for (TreeNode node : queue) {
                    result.add(node.val);
                }
                return result;
            }

            int size = queue.size();
            for (int i = 0; i &lt; size; i++) {
                TreeNode node = queue.poll();

                if (node.left != null &amp;&amp; !visited.contains(node.left)) {
                    queue.offer(node.left);
                    visited.add(node.left);
                }
                if (node.right != null &amp;&amp; !visited.contains(node.right)) {
                    queue.offer(node.right);
                    visited.add(node.right);
                }
                if (parent.get(node) != null &amp;&amp; !visited.contains(parent.get(node))) {
                    queue.offer(parent.get(node));
                    visited.add(parent.get(node));
                }
            }
            distance++;
        }

        return new ArrayList&lt;&gt;();
    }

    private void buildParentMap(TreeNode node, TreeNode par) {
        if (node == null) return;
        parent.put(node, par);
```

```
        buildParentMap(node.left, node);
        buildParentMap(node.right, node);
    }
}
```

5. Graphs

Problem 58: Number of Islands (Medium)

Description: Given a 2D grid of '1's (land) and '0's (water), count the number of islands.

Solution:

```
class Solution {
    public int numIslands(char[][] grid) {
        if (grid == null || grid.length == 0) return 0;

        int count = 0;
        for (int i = 0; i &lt; grid.length; i++) {
            for (int j = 0; j &lt; grid[0].length; j++) {
                if (grid[i][j] == '1') {
                    count++;
                    dfs(grid, i, j);
                }
            }
        }

        return count;
    }

    private void dfs(char[][] grid, int i, int j) {
        if (i &lt; 0 || i &gt;= grid.length || j &lt; 0 || j &gt;= grid[0].length ||
            grid[i][j] == '0') {
            return;
        }

        grid[i][j] = '0';
        dfs(grid, i + 1, j);
        dfs(grid, i - 1, j);
        dfs(grid, i, j + 1);
        dfs(grid, i, j - 1);
    }
}
```

Problem 59: Clone Graph (Medium)

Description: Clone an undirected graph where each node contains a value and a list of neighbors.

Solution:

```
class Solution {
    private Map&lt;Node, Node&gt; visited = new HashMap&lt;&gt;();

    public Node cloneGraph(Node node) {
        if (node == null) return null;

        if (visited.containsKey(node)) {
            return visited.get(node);
        }

        Node cloneNode = new Node(node.val, new ArrayList&lt;&gt;());
        visited.put(node, cloneNode);

        for (Node neighbor : node.neighbors) {
            cloneNode.neighbors.add(cloneGraph(neighbor));
        }
```

```
        return cloneNode;
    }
}
```

Problem 60: Course Schedule (Medium)

Description: Given prerequisites for courses, determine if you can finish all courses.

Solution:

```
class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        List&lt;Integer&gt;[] graph = new ArrayList[numCourses];
        int[] indegree = new int[numCourses];

        for (int i = 0; i &lt; numCourses; i++) {
            graph[i] = new ArrayList&lt;&gt;();
        }

        for (int[] prereq : prerequisites) {
            graph[prereq[1]].add(prereq[0]);
            indegree[prereq[0]]++;
        }

        Queue&lt;Integer&gt; queue = new LinkedList&lt;&gt;();
        for (int i = 0; i &lt; numCourses; i++) {
            if (indegree[i] == 0) {
                queue.offer(i);
            }
        }

        int count = 0;
        while (!queue.isEmpty()) {
            int course = queue.poll();
            count++;

            for (int next : graph[course]) {
                indegree[next]--;
                if (indegree[next] == 0) {
                    queue.offer(next);
                }
            }
        }

        return count == numCourses;
    }
}
```

Problem 61: Course Schedule II (Medium)

Description: Return the ordering of courses you should take to finish all courses.

Solution:

```
class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List&lt;Integer&gt;[] graph = new ArrayList[numCourses];
        int[] indegree = new int[numCourses];

        for (int i = 0; i &lt; numCourses; i++) {
            graph[i] = new ArrayList&lt;&gt;();
        }

        for (int[] prereq : prerequisites) {
            graph[prereq[1]].add(prereq[0]);
            indegree[prereq[0]]++;
        }
```

```
        Queue&lt;Integer&gt; queue = new LinkedList&lt;&gt;();
        for (int i = 0; i &lt; numCourses; i++) {
            if (indegree[i] == 0) {
                queue.offer(i);
            }
        }

        int[] result = new int[numCourses];
        int index = 0;

        while (!queue.isEmpty()) {
            int course = queue.poll();
            result[index++] = course;

            for (int next : graph[course]) {
                indegree[next]--;
                if (indegree[next] == 0) {
                    queue.offer(next);
                }
            }
        }

        return index == numCourses ? result : new int[0];
    }
}
```

Problem 62: Word Ladder (Hard)

Description: Transform a begin word to an end word using a dictionary, changing one letter at a time.

Solution:

```
class Solution {
    public int ladderLength(String beginWord, String endWord, List&lt;String&gt; wordList) {
        Set&lt;String&gt; wordSet = new HashSet&lt;&gt;(wordList);
        if (!wordSet.contains(endWord)) return 0;

        Queue&lt;String&gt; queue = new LinkedList&lt;&gt;();
        queue.offer(beginWord);
        int level = 1;

        while (!queue.isEmpty()) {
            int size = queue.size();

            for (int i = 0; i &lt; size; i++) {
                String word = queue.poll();
                char[] chars = word.toCharArray();

                for (int j = 0; j &lt; chars.length; j++) {
                    char original = chars[j];

                    for (char c = 'a'; c &lt;= 'z'; c++) {
                        if (c == original) continue;

                        chars[j] = c;
                        String newWord = new String(chars);

                        if (newWord.equals(endWord)) {
                            return level + 1;
                        }

                        if (wordSet.contains(newWord)) {
                            queue.offer(newWord);
                            wordSet.remove(newWord);
                        }
                    }

                    chars[j] = original;
                }
```

```
            }
            level++;
        }

        return 0;
    }
}
```

Problem 63: Pacific Atlantic Water Flow (Medium)

Description: Find coordinates where water can flow to both Pacific and Atlantic oceans.

Solution:

```
class Solution {
    private int[][] directions = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};

    public List&lt;List&lt;Integer&gt;&gt; pacificAtlantic(int[][] heights) {
        List&lt;List&lt;Integer&gt;&gt; result = new ArrayList&lt;&gt;();
        if (heights == null || heights.length == 0) return result;

        int m = heights.length, n = heights[0].length;
        boolean[][] pacific = new boolean[m][n];
        boolean[][] atlantic = new boolean[m][n];

        for (int i = 0; i &lt; m; i++) {
            dfs(heights, pacific, i, 0);
            dfs(heights, atlantic, i, n - 1);
        }

        for (int j = 0; j &lt; n; j++) {
            dfs(heights, pacific, 0, j);
            dfs(heights, atlantic, m - 1, j);
        }

        for (int i = 0; i &lt; m; i++) {
            for (int j = 0; j &lt; n; j++) {
                if (pacific[i][j] &amp;&amp; atlantic[i][j]) {
                    result.add(Arrays.asList(i, j));
                }
            }
        }

        return result;
    }

    private void dfs(int[][] heights, boolean[][] visited, int i, int j) {
        visited[i][j] = true;

        for (int[] dir : directions) {
            int x = i + dir[0];
            int y = j + dir[1];

            if (x &gt;= 0 &amp;&amp; x &lt; heights.length &amp;&amp; y &gt;= 0 &amp;&amp; y &lt; heights[0
                !visited[x][y] &amp;&amp; heights[x][y] &gt;= heights[i][j]) {
                dfs(heights, visited, x, y);
            }
        }
    }
}
```

Problem 64: Surrounded Regions (Medium)

Description: Capture all regions surrounded by 'X' in a 2D board.

Solution:

```
class Solution {
    public void solve(char[][] board) {
        if (board == null || board.length == 0) return;

        int m = board.length, n = board[0].length;

        // Mark O's connected to borders
        for (int i = 0; i &lt; m; i++) {
            if (board[i][0] == 'O') dfs(board, i, 0);
            if (board[i][n - 1] == 'O') dfs(board, i, n - 1);
        }

        for (int j = 0; j &lt; n; j++) {
            if (board[0][j] == 'O') dfs(board, 0, j);
            if (board[m - 1][j] == 'O') dfs(board, m - 1, j);
        }

        // Flip all O to X and all # back to O
        for (int i = 0; i &lt; m; i++) {
            for (int j = 0; j &lt; n; j++) {
                if (board[i][j] == 'O') {
                    board[i][j] = 'X';
                } else if (board[i][j] == '#') {
                    board[i][j] = 'O';
                }
            }
        }
    }

    private void dfs(char[][] board, int i, int j) {
        if (i &lt; 0 || i &gt;= board.length || j &lt; 0 || j &gt;= board[0].length ||
            board[i][j] != 'O') {
            return;
        }

        board[i][j] = '#';
        dfs(board, i + 1, j);
        dfs(board, i - 1, j);
        dfs(board, i, j + 1);
        dfs(board, i, j - 1);
    }
}
```

Problem 65: Graph Valid Tree (Medium)

Description: Given n nodes and edges, check if they form a valid tree.

Solution:

```
class Solution {
    public boolean validTree(int n, int[][] edges) {
        if (edges.length != n - 1) return false;

        List&lt;Integer&gt;[] graph = new ArrayList[n];
        for (int i = 0; i &lt; n; i++) {
            graph[i] = new ArrayList&lt;&gt;();
        }

        for (int[] edge : edges) {
            graph[edge[0]].add(edge[1]);
            graph[edge[1]].add(edge[0]);
        }
```

```
        boolean[] visited = new boolean[n];
        if (!dfs(graph, visited, 0, -1)) {
            return false;
        }

        for (boolean v : visited) {
            if (!v) return false;
        }

        return true;
    }

    private boolean dfs(List&lt;Integer&gt;[] graph, boolean[] visited,
                        int node, int parent) {
        visited[node] = true;

        for (int neighbor : graph[node]) {
            if (neighbor == parent) continue;
            if (visited[neighbor]) return false;
            if (!dfs(graph, visited, neighbor, node)) return false;
        }

        return true;
    }
}
```

Problem 66: Number of Connected Components (Medium)

Description: Find the number of connected components in an undirected graph.

Solution:

```
class Solution {
    public int countComponents(int n, int[][] edges) {
        List&lt;Integer&gt;[] graph = new ArrayList[n];
        for (int i = 0; i &lt; n; i++) {
            graph[i] = new ArrayList&lt;&gt;();
        }

        for (int[] edge : edges) {
            graph[edge[0]].add(edge[1]);
            graph[edge[1]].add(edge[0]);
        }

        boolean[] visited = new boolean[n];
        int count = 0;

        for (int i = 0; i &lt; n; i++) {
            if (!visited[i]) {
                dfs(graph, visited, i);
                count++;
            }
        }

        return count;
    }

    private void dfs(List&lt;Integer&gt;[] graph, boolean[] visited, int node) {
        visited[node] = true;

        for (int neighbor : graph[node]) {
            if (!visited[neighbor]) {
                dfs(graph, visited, neighbor);
            }
        }
    }
}
```

Problem 67: Redundant Connection (Medium)

Description: Find an edge that can be removed to make the graph a tree.

Solution:

```
class Solution {
    private int[] parent;

    public int[] findRedundantConnection(int[][] edges) {
        parent = new int[edges.length + 1];
        for (int i = 0; i &lt; parent.length; i++) {
            parent[i] = i;
        }

        for (int[] edge : edges) {
            if (!union(edge[0], edge[1])) {
                return edge;
            }
        }

        return new int[0];
    }

    private int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);
        }
        return parent[x];
    }

    private boolean union(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);

        if (rootX == rootY) return false;

        parent[rootX] = rootY;
        return true;
    }
}
```

Problem 68: Minimum Height Trees (Medium)

Description: Find all the root labels of minimum height trees.

Solution:

```
class Solution {
    public List&lt;Integer&gt; findMinHeightTrees(int n, int[][] edges) {
        if (n == 1) return Collections.singletonList(0);

        List&lt;Set&lt;Integer&gt;&gt; graph = new ArrayList&lt;&gt;();
        for (int i = 0; i &lt; n; i++) {
            graph.add(new HashSet&lt;&gt;());
        }

        for (int[] edge : edges) {
            graph.get(edge[0]).add(edge[1]);
            graph.get(edge[1]).add(edge[0]);
        }

        List&lt;Integer&gt; leaves = new ArrayList&lt;&gt;();
        for (int i = 0; i &lt; n; i++) {
            if (graph.get(i).size() == 1) {
                leaves.add(i);
            }
        }
```

```
        int remaining = n;
        while (remaining &gt; 2) {
            remaining -= leaves.size();
            List&lt;Integer&gt; newLeaves = new ArrayList&lt;&gt;();

            for (int leaf : leaves) {
                int neighbor = graph.get(leaf).iterator().next();
                graph.get(neighbor).remove(leaf);

                if (graph.get(neighbor).size() == 1) {
                    newLeaves.add(neighbor);
                }
            }

            leaves = newLeaves;
        }

        return leaves;
    }
}
```

Problem 69: Alien Dictionary (Hard)

Description: Derive the order of letters in an alien language from sorted words.

Solution:

```
class Solution {
    public String alienOrder(String[] words) {
        Map&lt;Character, Set&lt;Character&gt;&gt; graph = new HashMap&lt;&gt;();
        Map&lt;Character, Integer&gt; indegree = new HashMap&lt;&gt;();

        for (String word : words) {
            for (char c : word.toCharArray()) {
                graph.putIfAbsent(c, new HashSet&lt;&gt;());
                indegree.putIfAbsent(c, 0);
            }
        }

        for (int i = 0; i &lt; words.length - 1; i++) {
            String word1 = words[i];
            String word2 = words[i + 1];
            int minLen = Math.min(word1.length(), word2.length());

            if (word1.length() &gt; word2.length() &amp;&amp;
                word1.startsWith(word2)) {
                return "";
            }

            for (int j = 0; j &lt; minLen; j++) {
                char c1 = word1.charAt(j);
                char c2 = word2.charAt(j);

                if (c1 != c2) {
                    if (!graph.get(c1).contains(c2)) {
                        graph.get(c1).add(c2);
                        indegree.put(c2, indegree.get(c2) + 1);
                    }
                    break;
                }
            }
        }

        Queue&lt;Character&gt; queue = new LinkedList&lt;&gt;();
        for (char c : indegree.keySet()) {
            if (indegree.get(c) == 0) {
                queue.offer(c);
            }
        }
```

```
        StringBuilder result = new StringBuilder();
        while (!queue.isEmpty()) {
            char c = queue.poll();
            result.append(c);

            for (char next : graph.get(c)) {
                indegree.put(next, indegree.get(next) - 1);
                if (indegree.get(next) == 0) {
                    queue.offer(next);
                }
            }
        }

        return result.length() == indegree.size() ? result.toString() : "";
    }
}
```

6. Heaps & Priority Queues

Problem 70: Kth Largest Element in Array (Medium)

Description: Find the kth largest element in an unsorted array.

Solution:

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        PriorityQueue&lt;Integer&gt; minHeap = new PriorityQueue&lt;&gt;();

        for (int num : nums) {
            minHeap.offer(num);
            if (minHeap.size() &gt; k) {
                minHeap.poll();
            }
        }

        return minHeap.peek();
    }
}
```

Problem 71: Top K Frequent Elements (Medium)

Description: Given an integer array, return the k most frequent elements.

Solution:

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map&lt;Integer, Integer&gt; freq = new HashMap&lt;&gt;();
        for (int num : nums) {
            freq.put(num, freq.getOrDefault(num, 0) + 1);
        }

        PriorityQueue&lt;Integer&gt; heap = new PriorityQueue&lt;&gt;(
            (a, b) -&gt; freq.get(a) - freq.get(b)
        );

        for (int num : freq.keySet()) {
            heap.offer(num);
            if (heap.size() &gt; k) {
                heap.poll();
            }
        }

        int[] result = new int[k];
        for (int i = 0; i &lt; k; i++) {
```

```
            result[i] = heap.poll();
        }

        return result;
    }
}
```

Problem 72: Find Median from Data Stream (Hard)

Description: Design a data structure that supports adding numbers and finding the median.

Solution:

```
class MedianFinder {
    private PriorityQueue&lt;Integer&gt; maxHeap;
    private PriorityQueue&lt;Integer&gt; minHeap;

    public MedianFinder() {
        maxHeap = new PriorityQueue&lt;&gt;((a, b) -&gt; b - a);
        minHeap = new PriorityQueue&lt;&gt;();
    }

    public void addNum(int num) {
        maxHeap.offer(num);
        minHeap.offer(maxHeap.poll());

        if (maxHeap.size() &lt; minHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }

    public double findMedian() {
        if (maxHeap.size() == minHeap.size()) {
            return (maxHeap.peek() + minHeap.peek()) / 2.0;
        }
        return maxHeap.peek();
    }
}
```

Problem 73: K Closest Points to Origin (Medium)

Description: Find the K closest points to the origin (0, 0).

Solution:

```
class Solution {
    public int[][] kClosest(int[][] points, int k) {
        PriorityQueue&lt;int[]&gt; maxHeap = new PriorityQueue&lt;&gt;(
            (a, b) -&gt; (b[0] * b[0] + b[1] * b[1]) - (a[0] * a[0] + a[1] * a[1])
        );

        for (int[] point : points) {
            maxHeap.offer(point);
            if (maxHeap.size() &gt; k) {
                maxHeap.poll();
            }
        }

        int[][] result = new int[k][2];
        for (int i = 0; i &lt; k; i++) {
            result[i] = maxHeap.poll();
        }

        return result;
    }
}
```

Problem 74: Meeting Rooms II (Medium)

Description: Find the minimum number of conference rooms required.

Solution:

```
class Solution {
    public int minMeetingRooms(int[][] intervals) {
        if (intervals.length == 0) return 0;

        Arrays.sort(intervals, (a, b) -&gt; a[0] - b[0]);
        PriorityQueue&lt;Integer&gt; heap = new PriorityQueue&lt;&gt;();

        heap.offer(intervals[0][1]);

        for (int i = 1; i &lt; intervals.length; i++) {
            if (intervals[i][0] &gt;= heap.peek()) {
                heap.poll();
            }
            heap.offer(intervals[i][1]);
        }

        return heap.size();
    }
}
```

Problem 75: Task Scheduler (Medium)

Description: Given tasks and a cooldown period n, find the minimum time to complete all tasks.

Solution:

```
class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] freq = new int[26];
        for (char task : tasks) {
            freq[task - 'A']++;
        }

        PriorityQueue&lt;Integer&gt; maxHeap = new PriorityQueue&lt;&gt;((a, b) -&gt; b - a);
        for (int f : freq) {
            if (f &gt; 0) {
                maxHeap.offer(f);
            }
        }

        int time = 0;
        Queue&lt;int[]&gt; queue = new LinkedList&lt;&gt;();

        while (!maxHeap.isEmpty() || !queue.isEmpty()) {
            time++;

            if (!maxHeap.isEmpty()) {
                int count = maxHeap.poll() - 1;
                if (count &gt; 0) {
                    queue.offer(new int[]{count, time + n});
                }
            }

            if (!queue.isEmpty() &amp;&amp; queue.peek()[1] == time) {
                maxHeap.offer(queue.poll()[0]);
            }
        }

        return time;
    }
}
```

Problem 76: Reorganize String (Medium)

Description: Rearrange string so no two adjacent characters are the same.

Solution:

```
class Solution {
    public String reorganizeString(String s) {
        int[] freq = new int[26];
        for (char c : s.toCharArray()) {
            freq[c - 'a']++;
        }

        PriorityQueue&lt;int[]&gt; maxHeap = new PriorityQueue&lt;&gt;((a, b) -&gt; b[1] - a[1]);
        for (int i = 0; i &lt; 26; i++) {
            if (freq[i] &gt; 0) {
                maxHeap.offer(new int[]{i, freq[i]});
            }
        }

        StringBuilder result = new StringBuilder();
        int[] prev = new int[]{-1, 0};

        while (!maxHeap.isEmpty()) {
            int[] current = maxHeap.poll();
            result.append((char)(current[0] + 'a'));

            if (prev[1] &gt; 0) {
                maxHeap.offer(prev);
            }

            current[1]--;
            prev = current;
        }

        return result.length() == s.length() ? result.toString() : "";
    }
}
```

Problem 77: Kth Smallest Element in Sorted Matrix (Medium)

Description: Given an n x n matrix where rows and columns are sorted, find the kth smallest element.

Solution:

```
class Solution {
    public int kthSmallest(int[][] matrix, int k) {
        int n = matrix.length;
        PriorityQueue&lt;int[]&gt; minHeap = new PriorityQueue&lt;&gt;(
            (a, b) -&gt; a[0] - b[0]
        );

        for (int i = 0; i &lt; Math.min(n, k); i++) {
            minHeap.offer(new int[]{matrix[i][0], i, 0});
        }

        int result = 0;
        for (int i = 0; i &lt; k; i++) {
            int[] current = minHeap.poll();
            result = current[0];
            int row = current[1];
            int col = current[2];

            if (col + 1 &lt; n) {
                minHeap.offer(new int[]{matrix[row][col + 1], row, col + 1});
            }
        }

        return result;
```

```
    }
}
```

7. Hash Tables & Sets

Problem 78: Two Sum (Medium)

Description: Given an array of integers, return indices of the two numbers that add up to a specific target.

Solution:

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map&lt;Integer, Integer&gt; map = new HashMap&lt;&gt;();

        for (int i = 0; i &lt; nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[]{map.get(complement), i};
            }
            map.put(nums[i], i);
        }

        return new int[0];
    }
}
```

Problem 79: Group Anagrams (Medium)

Description: Given an array of strings, group anagrams together.

Solution:

```
class Solution {
    public List&lt;List&lt;String&gt;&gt; groupAnagrams(String[] strs) {
        Map&lt;String, List&lt;String&gt;&gt; map = new HashMap&lt;&gt;();

        for (String str : strs) {
            char[] chars = str.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);

            map.putIfAbsent(key, new ArrayList&lt;&gt;());
            map.get(key).add(str);
        }

        return new ArrayList&lt;&gt;(map.values());
    }
}
```

Problem 80: Longest Substring Without Repeating Characters (Medium)

Description: Find the length of the longest substring without repeating characters.

Solution:

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Map&lt;Character, Integer&gt; map = new HashMap&lt;&gt;();
        int maxLen = 0;
        int left = 0;

        for (int right = 0; right &lt; s.length(); right++) {
            char c = s.charAt(right);
```

```
            if (map.containsKey(c)) {
                left = Math.max(left, map.get(c) + 1);
            }

            map.put(c, right);
            maxLen = Math.max(maxLen, right - left + 1);
        }

        return maxLen;
    }
}
```

Problem 81: Minimum Window Substring (Hard)

Description: Find the minimum window in string s which contains all characters of string t.

Solution:

```
class Solution {
    public String minWindow(String s, String t) {
        if (s.length() &lt; t.length()) return "";

        Map&lt;Character, Integer&gt; target = new HashMap&lt;&gt;();
        for (char c : t.toCharArray()) {
            target.put(c, target.getOrDefault(c, 0) + 1);
        }

        Map&lt;Character, Integer&gt; window = new HashMap&lt;&gt;();
        int left = 0, right = 0;
        int required = target.size();
        int formed = 0;
        int[] result = {-1, 0, 0};

        while (right &lt; s.length()) {
            char c = s.charAt(right);
            window.put(c, window.getOrDefault(c, 0) + 1);

            if (target.containsKey(c) &amp;&amp;
                window.get(c).intValue() == target.get(c).intValue()) {
                formed++;
            }

            while (left &lt;= right &amp;&amp; formed == required) {
                if (result[0] == -1 || right - left + 1 &lt; result[0]) {
                    result[0] = right - left + 1;
                    result[1] = left;
                    result[2] = right;
                }

                char leftChar = s.charAt(left);
                window.put(leftChar, window.get(leftChar) - 1);

                if (target.containsKey(leftChar) &amp;&amp;
                    window.get(leftChar) &lt; target.get(leftChar)) {
                    formed--;
                }

                left++;
            }

            right++;
        }

        return result[0] == -1 ? "" : s.substring(result[1], result[2] + 1);
    }
}
```

Problem 82: LRU Cache (Medium)

Description: Design a data structure for Least Recently Used (LRU) cache.

Solution:

```
class LRUCache {
    private class Node {
        int key, value;
        Node prev, next;
        Node(int k, int v) { key = k; value = v; }
    }

    private Map&lt;Integer, Node&gt; cache;
    private int capacity;
    private Node head, tail;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        cache = new HashMap&lt;&gt;();
        head = new Node(0, 0);
        tail = new Node(0, 0);
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        if (!cache.containsKey(key)) return -1;

        Node node = cache.get(key);
        remove(node);
        insert(node);
        return node.value;
    }

    public void put(int key, int value) {
        if (cache.containsKey(key)) {
            remove(cache.get(key));
        }

        Node node = new Node(key, value);
        cache.put(key, node);
        insert(node);

        if (cache.size() &gt; capacity) {
            Node lru = head.next;
            remove(lru);
            cache.remove(lru.key);
        }
    }

    private void remove(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void insert(Node node) {
        node.prev = tail.prev;
        node.next = tail;
        tail.prev.next = node;
        tail.prev = node;
    }
}
```

Problem 83: Design HashMap (Medium)

Description: Design a HashMap without using built-in hash table libraries.

Solution:

```
class MyHashMap {
    private class Node {
        int key, value;
        Node next;
        Node(int k, int v) { key = k; value = v; }
    }

    private static final int SIZE = 1000;
    private Node[] buckets;

    public MyHashMap() {
        buckets = new Node[SIZE];
    }

    private int hash(int key) {
        return key % SIZE;
    }

    public void put(int key, int value) {
        int index = hash(key);

        if (buckets[index] == null) {
            buckets[index] = new Node(key, value);
            return;
        }

        Node current = buckets[index];
        while (true) {
            if (current.key == key) {
                current.value = value;
                return;
            }
            if (current.next == null) break;
            current = current.next;
        }
        current.next = new Node(key, value);
    }

    public int get(int key) {
        int index = hash(key);
        Node current = buckets[index];

        while (current != null) {
            if (current.key == key) {
                return current.value;
            }
            current = current.next;
        }

        return -1;
    }

    public void remove(int key) {
        int index = hash(key);
        Node current = buckets[index];

        if (current == null) return;

        if (current.key == key) {
            buckets[index] = current.next;
            return;
        }

        while (current.next != null) {
            if (current.next.key == key) {
```

```
                current.next = current.next.next;
                return;
            }
            current = current.next;
        }
    }
}
```

Problem 84: Isomorphic Strings (Medium)

Description: Determine if two strings are isomorphic.

Solution:

```
class Solution {
    public boolean isIsomorphic(String s, String t) {
        if (s.length() != t.length()) return false;

        Map&lt;Character, Character&gt; mapS = new HashMap&lt;&gt;();
        Map&lt;Character, Character&gt; mapT = new HashMap&lt;&gt;();

        for (int i = 0; i &lt; s.length(); i++) {
            char c1 = s.charAt(i);
            char c2 = t.charAt(i);

            if (mapS.containsKey(c1)) {
                if (mapS.get(c1) != c2) return false;
            } else {
                mapS.put(c1, c2);
            }

            if (mapT.containsKey(c2)) {
                if (mapT.get(c2) != c1) return false;
            } else {
                mapT.put(c2, c1);
            }
        }

        return true;
    }
}
```

Problem 85: Valid Sudoku (Medium)

Description: Determine if a 9x9 Sudoku board is valid.

Solution:

```
class Solution {
    public boolean isValidSudoku(char[][] board) {
        Set&lt;String&gt; seen = new HashSet&lt;&gt;();

        for (int i = 0; i &lt; 9; i++) {
            for (int j = 0; j &lt; 9; j++) {
                char c = board[i][j];
                if (c != '.') {
                    if (!seen.add(c + " in row " + i) ||
                        !seen.add(c + " in col " + j) ||
                        !seen.add(c + " in box " + i/3 + "-" + j/3)) {
                        return false;
                    }
                }
            }
        }

        return true;
```

```
    }
}
```

Problem 86: Contains Duplicate II (Medium)

Description: Given an array and an integer k, find if there are two distinct indices i and j such that nums[i] = nums[j] and abs(i - j) <= k.

Solution:

```
class Solution {
    public boolean containsNearbyDuplicate(int[] nums, int k) {
        Map&lt;Integer, Integer&gt; map = new HashMap&lt;&gt;();

        for (int i = 0; i &lt; nums.length; i++) {
            if (map.containsKey(nums[i])) {
                if (i - map.get(nums[i]) &lt;= k) {
                    return true;
                }
            }
            map.put(nums[i], i);
        }

        return false;
    }
}
```

Problem 87: 4Sum II (Medium)

Description: Given four arrays, compute how many tuples (i, j, k, l) there are such that A[i] + B[j] + C[k] + D[l] is zero.

Solution:

```
class Solution {
    public int fourSumCount(int[] nums1, int[] nums2, int[] nums3, int[] nums4) {
        Map&lt;Integer, Integer&gt; map = new HashMap&lt;&gt;();

        for (int a : nums1) {
            for (int b : nums2) {
                map.put(a + b, map.getOrDefault(a + b, 0) + 1);
            }
        }

        int count = 0;
        for (int c : nums3) {
            for (int d : nums4) {
                count += map.getOrDefault(-(c + d), 0);
            }
        }

        return count;
    }
}
```

8. Dynamic Programming with Data Structures

Problem 88: House Robber II (Medium)

Description: Houses are arranged in a circle. Find maximum amount you can rob without robbing adjacent houses.

Solution:

```
class Solution {
    public int rob(int[] nums) {
```

```
        if (nums.length == 1) return nums[0];

        return Math.max(
            robLinear(nums, 0, nums.length - 2),
            robLinear(nums, 1, nums.length - 1)
        );
    }

    private int robLinear(int[] nums, int start, int end) {
        int prev2 = 0, prev1 = 0;

        for (int i = start; i &lt;= end; i++) {
            int temp = prev1;
            prev1 = Math.max(prev1, prev2 + nums[i]);
            prev2 = temp;
        }

        return prev1;
    }
}
```

Problem 89: Word Break (Medium)

Description: Given a string and a dictionary, determine if the string can be segmented into words from the dictionary.

Solution:

```
class Solution {
    public boolean wordBreak(String s, List&lt;String&gt; wordDict) {
        Set&lt;String&gt; wordSet = new HashSet&lt;&gt;(wordDict);
        boolean[] dp = new boolean[s.length() + 1];
        dp[0] = true;

        for (int i = 1; i &lt;= s.length(); i++) {
            for (int j = 0; j &lt; i; j++) {
                if (dp[j] &amp;&amp; wordSet.contains(s.substring(j, i))) {
                    dp[i] = true;
                    break;
                }
            }
        }

        return dp[s.length()];
    }
}
```

Problem 90: Coin Change (Medium)

Description: Find the minimum number of coins needed to make up a given amount.

Solution:

```
class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;

        for (int i = 1; i &lt;= amount; i++) {
            for (int coin : coins) {
                if (coin &lt;= i) {
                    dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                }
            }
        }

        return dp[amount] &gt; amount ? -1 : dp[amount];
```

```
    }
}
```

Problem 91: Longest Increasing Subsequence (Medium)

Description: Find the length of the longest strictly increasing subsequence.

Solution:

```
class Solution {
    public int lengthOfLIS(int[] nums) {
        List&lt;Integer&gt; dp = new ArrayList&lt;&gt;();

        for (int num : nums) {
            int pos = Collections.binarySearch(dp, num);
            if (pos &lt; 0) {
                pos = -(pos + 1);
            }

            if (pos == dp.size()) {
                dp.add(num);
            } else {
                dp.set(pos, num);
            }
        }

        return dp.size();
    }
}
```

Problem 92: Unique Paths II (Medium)

Description: Find number of unique paths in a grid with obstacles.

Solution:

```
class Solution {
    public int uniquePathsWithObstacles(int[][] obstacleGrid) {
        int m = obstacleGrid.length;
        int n = obstacleGrid[0].length;

        if (obstacleGrid[0][0] == 1) return 0;

        int[][] dp = new int[m][n];
        dp[0][0] = 1;

        for (int i = 1; i &lt; m; i++) {
            dp[i][0] = (obstacleGrid[i][0] == 0 &amp;&amp; dp[i-1][0] == 1) ? 1 : 0;
        }

        for (int j = 1; j &lt; n; j++) {
            dp[0][j] = (obstacleGrid[0][j] == 0 &amp;&amp; dp[0][j-1] == 1) ? 1 : 0;
        }

        for (int i = 1; i &lt; m; i++) {
            for (int j = 1; j &lt; n; j++) {
                if (obstacleGrid[i][j] == 0) {
                    dp[i][j] = dp[i-1][j] + dp[i][j-1];
                }
            }
        }

        return dp[m-1][n-1];
    }
}
```

Problem 93: Jump Game II (Medium)

Description: Find the minimum number of jumps to reach the last index.

Solution:

```
class Solution {
    public int jump(int[] nums) {
        int jumps = 0;
        int currentEnd = 0;
        int farthest = 0;

        for (int i = 0; i &lt; nums.length - 1; i++) {
            farthest = Math.max(farthest, i + nums[i]);

            if (i == currentEnd) {
                jumps++;
                currentEnd = farthest;
            }
        }

        return jumps;
    }
}
```

Problem 94: Maximal Square (Medium)

Description: Find the largest square containing only 1's in a binary matrix.

Solution:

```
class Solution {
    public int maximalSquare(char[][] matrix) {
        if (matrix.length == 0) return 0;

        int m = matrix.length;
        int n = matrix[0].length;
        int[][] dp = new int[m + 1][n + 1];
        int maxSide = 0;

        for (int i = 1; i &lt;= m; i++) {
            for (int j = 1; j &lt;= n; j++) {
                if (matrix[i-1][j-1] == '1') {
                    dp[i][j] = Math.min(Math.min(dp[i-1][j], dp[i][j-1]),
                                        dp[i-1][j-1]) + 1;
                    maxSide = Math.max(maxSide, dp[i][j]);
                }
            }
        }

        return maxSide * maxSide;
    }
}
```

Problem 95: Partition Equal Subset Sum (Medium)

Description: Determine if array can be partitioned into two subsets with equal sum.

Solution:

```
class Solution {
    public boolean canPartition(int[] nums) {
        int sum = 0;
        for (int num : nums) {
            sum += num;
        }
```

```
        if (sum % 2 != 0) return false;

        int target = sum / 2;
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;

        for (int num : nums) {
            for (int j = target; j &gt;= num; j--) {
                dp[j] = dp[j] || dp[j - num];
            }
        }

        return dp[target];
    }
}
```

Problem 96: Decode Ways (Medium)

Description: Count the number of ways to decode a digit string to letters (A=1, B=2, ..., Z=26).

Solution:

```
class Solution {
    public int numDecodings(String s) {
        if (s.charAt(0) == '0') return 0;

        int n = s.length();
        int[] dp = new int[n + 1];
        dp[0] = 1;
        dp[1] = 1;

        for (int i = 2; i &lt;= n; i++) {
            int oneDigit = Integer.parseInt(s.substring(i - 1, i));
            int twoDigits = Integer.parseInt(s.substring(i - 2, i));

            if (oneDigit &gt;= 1) {
                dp[i] += dp[i - 1];
            }

            if (twoDigits &gt;= 10 &amp;&amp; twoDigits &lt;= 26) {
                dp[i] += dp[i - 2];
            }
        }

        return dp[n];
    }
}
```

Problem 97: Target Sum (Medium)

Description: Find number of ways to assign symbols to make sum equal to target.

Solution:

```
class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int sum = 0;
        for (int num : nums) {
            sum += num;
        }

        if (Math.abs(target) &gt; sum || (sum + target) % 2 != 0) {
            return 0;
        }

        int subsetSum = (sum + target) / 2;
        int[] dp = new int[subsetSum + 1];
        dp[0] = 1;
```

```
        for (int num : nums) {
            for (int j = subsetSum; j &gt;= num; j--) {
                dp[j] += dp[j - num];
            }
        }

        return dp[subsetSum];
    }
}
```

Problem 98: Longest Palindromic Substring (Medium)

Description: Find the longest palindromic substring in a string.

Solution:

```
class Solution {
    public String longestPalindrome(String s) {
        if (s.length() &lt; 2) return s;

        int start = 0, maxLen = 0;

        for (int i = 0; i &lt; s.length(); i++) {
            int len1 = expandFromCenter(s, i, i);
            int len2 = expandFromCenter(s, i, i + 1);
            int len = Math.max(len1, len2);

            if (len &gt; maxLen) {
                maxLen = len;
                start = i - (len - 1) / 2;
            }
        }

        return s.substring(start, start + maxLen);
    }

    private int expandFromCenter(String s, int left, int right) {
        while (left &gt;= 0 &amp;&amp; right &lt; s.length() &amp;&amp;
               s.charAt(left) == s.charAt(right)) {
            left--;
            right++;
        }
        return right - left - 1;
    }
}
```

Problem 99: Palindrome Partitioning (Medium)

Description: Partition string such that every substring is a palindrome. Return all possible partitions.

Solution:

```
class Solution {
    public List&lt;List&lt;String&gt;&gt; partition(String s) {
        List&lt;List&lt;String&gt;&gt; result = new ArrayList&lt;&gt;();
        backtrack(s, 0, new ArrayList&lt;&gt;(), result);
        return result;
    }

    private void backtrack(String s, int start, List&lt;String&gt; current,
                           List&lt;List&lt;String&gt;&gt; result) {
        if (start == s.length()) {
            result.add(new ArrayList&lt;&gt;(current));
            return;
        }

        for (int end = start + 1; end &lt;= s.length(); end++) {
```

```
            String substring = s.substring(start, end);
            if (isPalindrome(substring)) {
                current.add(substring);
                backtrack(s, end, current, result);
                current.remove(current.size() - 1);
            }
        }
    }

    private boolean isPalindrome(String s) {
        int left = 0, right = s.length() - 1;
        while (left &lt; right) {
            if (s.charAt(left++) != s.charAt(right--)) {
                return false;
            }
        }
        return true;
    }
}
```

Problem 100: Regular Expression Matching (Hard)

Description: Implement regular expression matching with support for '.' and '*'.

Solution:

```
class Solution {
    public boolean isMatch(String s, String p) {
        int m = s.length();
        int n = p.length();
        boolean[][] dp = new boolean[m + 1][n + 1];

        dp[0][0] = true;

        for (int j = 2; j &lt;= n; j++) {
            if (p.charAt(j - 1) == '*') {
                dp[0][j] = dp[0][j - 2];
            }
        }

        for (int i = 1; i &lt;= m; i++) {
            for (int j = 1; j &lt;= n; j++) {
                char sc = s.charAt(i - 1);
                char pc = p.charAt(j - 1);

                if (pc == '.' || pc == sc) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else if (pc == '*') {
                    char prevChar = p.charAt(j - 2);
                    dp[i][j] = dp[i][j - 2];

                    if (prevChar == '.' || prevChar == sc) {
                        dp[i][j] = dp[i][j] || dp[i - 1][j];
                    }
                }
            }
        }

        return dp[m][n];
    }
}
```

Conclusion

This collection covers 100 medium to hard LeetCode problems focused on essential data structures in Java. These problems span:

- **Arrays**: Searching, sorting, two-pointers, sliding window
- **Linked Lists**: Reversal, cycle detection, merging
- **Stacks & Queues**: Monotonic stack, deque operations
- **Trees**: Traversals, BST operations, path problems
- **Graphs**: DFS, BFS, topological sort, union-find
- **Heaps**: Top K elements, median finding, scheduling
- **Hash Tables**: Fast lookups, caching, frequency counting
- **Dynamic Programming**: Optimization problems with arrays

Practice Tips:

1. Understand the problem thoroughly before coding
2. Draw examples and edge cases
3. Think about time and space complexity
4. Code iteratively and test frequently
5. Review multiple solutions for each problem

Good luck with your interview preparation!